IPA Course Project

Processor Architecture Design based on the RISC-V ISA using Verilog

Sajiv Singh
Roll No: 2023112003

Krrish Goenka
Roll No: 2023112023

Pa Kiruba
Roll No: 2023112010

***Abstract*—Creating a simple processor based on the RISCV instruction set. Its functionalities include basic arithmetic, logical, memory and branching operations. Implemented on Icarus Verilog and tested using GTKWave.**

THE AIM OF THIS PROJECT is to design a structural processor design in Verilog based on RISC-V instruction set architecture. The implemented processor performs basic functions such as addition, subtraction, bitwise AND, bitwise OR, branch if equal, load a double word to a register from memory and store a double word into the data memory. The above-mentioned instructions have been implemented in two ways, one being sequential and the other pipelined. Each of these has their own merits and demerits, which are discussed in detail in the report, along with their implementation.

Introduction

Processor architecture design is the foundation of modern computing, shaping how hardware executes instructions and processes data. It defines the organization and operation of the components of a processor, determining factors such as speed, efficiency, and scalability. A well-designed architecture is critical to balancing performance and power consumption. Key design considerations include choosing between the RISC (Reduced Instruction Set Computing) and CISC (Complex Instruction Set Computing) paradigms. The project comprises a RISC Based ISA, which manages data flow and control signals, and handles memory hierarchies for fast, efficient access to data.

The processor implementation includes a subset of the RISC-V instruction set, i.e. the memory reference instructions load doubleword (ld) and store doubleword (sd), the arithmetic-logical instructions AND, OR, ADD, SUB, and the conditional branch if equal instruction (beq). The processor itself consists mainly of the primary counter (PC), instruction memory, register file, arithmetic logic unit (ALU), and data memory. Based on the instruction, some components are used (some are not) and some may run a different process. Due to the inherent disadvantages of a single-cycle implementation, we further proceed to make a pipeline design that retains the great ideas of design. By pipelining, we achieve greater throughput provided we detect and counter the possible pipelining hazards.

Published by the IEEE Consumer Technology Society

SEQUENTIAL IMPLEMENTATION

An Overview of the Implementation

For any instruction, the processor sends the *program counter (PC)* to the memory that contains the code, fetches the instruction from that memory, reads one or two registers (using fields of the instruction to select the registers to read). For the ld instruction, we need to read only one register, but most other instructions require reading two registers. After these two steps, the actions required to complete the instruction depend on the instruction class (covered in detail in later sections).

The *ALU* is used by all instruction classes for different purposes. The memory reference instructions use the ALU for an address calculation, the arithmetic-logical instructions for the operation execution, and conditional branches for the equality test for beq (via subtraction). It is imperative to utilize the same ALU for different instructions, thus introducing the need to feed it information of the same. A *control unit* allows us to determine how a unit (not only the ALU but other units like memory as well, as seen later) must function based on the instruction inputted to it.

Stages of the Sequential Processor

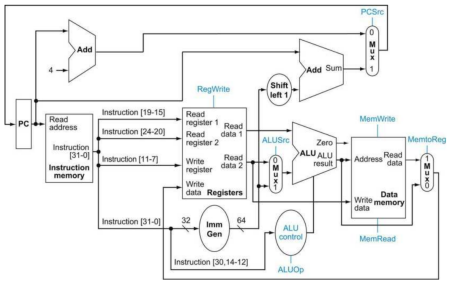

Figure 1. The datapath of a Sequential processor with all control lines

There are broadly 4 stages to a processor. The **instruction memory** reads the address in PC and identifies the instruction format. Based on the instruction format, the corresponding bit sequences are assigned to each input of the **register file** and other units. If not an R-type instruction, an immediate is also calculated by the **immediate generator**. The **data memory** handles any data that is to be read from or written to. Unlike the other units whose write is controlled by a control signal, here both read and write are controlled by control signals (since we'll be working with store instructions). The **control unit** drives each unit to perform a task via the control signals. One such example other than the ALU is the data memory, which is read or written based on the control signal *MemRead* and *MemWrite* respectively. There also exists the ALU control which drives the ALU to perform one of its 4 functionalities (AND, OR, add, sub). The register file is written only when the control signal *RegWrite* is asserted. The select lines of all muxes are control signals too.

The PC, which is a 64-bit register whose role is to contain the address of the successive instruction. The PC is updated via 2 ways. One is a normal increment of 4 bytes in the memory to go to the next instruction. The other is when the branch target is addressed, i.e, jumping to a target branch. In the latter, the offset is PC-related and is calculated via the immediate (as will be seen later). Based on the instruction, a mux is used to select one of the above two, which is yet again driven by a control signal *PCSrc*.

Reading a register and writing onto it is done at the register file. It reads data stored in the register whose address is at *ReadRegister1* and *Readregister2* and outputs them at *ReadData1* and *ReadData2* respectively. If the control signal *RegWrite* is asserted, data received via *WriteData* is written at the *rd* address provided to *WriteRegister*.

Immediate generator (*Immgen*) receives the entire 32-bit instruction, parses the immediate embedded in it (which is of 12 or 20 bits), sign-extends it to 64 bits and left shifts by 1. Parsing of the instruction is again performed using muxes. Certain bits in the opcode (5th and 6th) are used to identify the instruction type, thus the location of the immediate bits. We use these opcode bits as select lines to muxes fed with immediate bits and construct the immediate before sign-extending. This left shift not only increases the range of the offset that can be incremented onto the PC, but also allows us to support instructions involving halfwords (like lh, which we wouldn't be able to do if had left shifted twice in the pursuit of greater range). Immediate before sign-extending is smaller than 64-bits, hence, we don't lose the MSB by left-shifting once.

Unlike register file which reads by default and only requires write access (via the control signal *RegWrite*), data memory requires both read and write access as both load and store commands use it. In our implementation (positive-edge based), we write onto memories at the negative-edge of the clock. To do so,

we had to use a mux to set the PC to zero to initialize it.

The ALU calculates the offset from immediate and *adds* it to a register address. R-type instructions are performed with their corresponding dedicated operation. The branching condition for beq is checked by subtracting the data in the two registers and comparing their difference with x0 (hard-wired to 0).

Testing and Output

After testing our code on multiple testbenches, we found it to work as expected. Attached below is one example encompassing all functions and edge cases followed by the GTKWave output for the reader to verify.

```
## TESTCASE 1
//memory has values 3,2,5 on
    successive locations
ld  x4  0(x0)  : 0
ld  x5  8(x0)  : 4
ld  x6  16(x0) : 8
beq x4  x5  label1[branch not taken]
add x5  x4  x5  : 16
beq x5  x6  label2
add x0  x0  x0  : 24
Sub x0  x0  x0  : 28
Sub x7  x6  x4  : 32
AND x8  x7  x4  : 36
OR  x9  x4  x0  : 40
SD  x9  24(x0) : 44
```

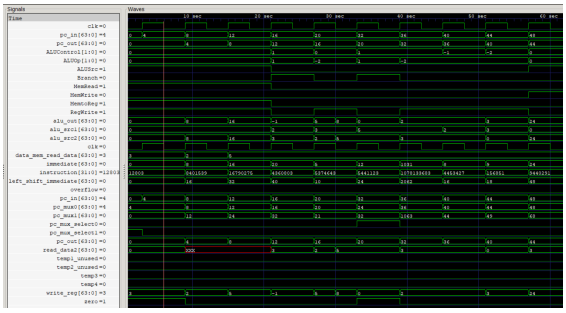

Figure 2. GTKWave output of sequential implementation

We shall now dissect the code and discuss the working of the blocks involved in each line. As briefly mentioned before, the PC is initialized to zero. Since we are writing onto the memory in the negative edge and Verilog, by default assumes a positive edge at t=0, we must set the PC to zero for the second cycle as well. This is done using a mux's select line. The instruction memory reads the address of the instruction in the instruction memory and pushes it out to the register file. We're loading the registers x4, x5 and x6 with the data in x0, x8 and x16 of the data memory respectively. The offsets 0 and 8 must be calculated at the ALU which has been written considering a 64-bit input. Using the Immgen block, we sign-extend and leftshift the offset by 1 and push this output to the ALU. The ALU block now calculates the offset and feeds it to the data memory. The control signal *MemRead* is asserted allowing data memory to access the data using the address from ALU. *MemtoReg* and *RegWrite* are now asserted to allow the writing of this data from ReadData to register file and writing onto the corresponding registers x4, x5 and x6. In the next instruction (beq), immgen again comes into play as we have to calculate the offset and add it to the PC. A *zero* flag is de-asserted since x4 and x5 are unequal forcing the PC to be updated with an increment of 4. The ALU result is neglected due to *MemWrite* being zero. The add instruction works similar to above, except this time, the control signal *ALUSrc* is de-asserted, neglecting the immgen output. The ALU is used to calculate the sum of the values stored in rs1 and rs2, which overwrites the register file (*MemtoReg* is deasserted, blocking off the path to data memory while the *RegWrite* is asserted). The condition test is satisfied in the successive beq instruction, asserting the *zero* flag and the *Branch* control signal. This allows the PC to be incremented by the offset calculated at the ALU instead of a regular increment by 4. The remaining instructions follow the same behavior as above.

PIPELINED IMPLEMENTATION

Need for Pipelining

In case of sequential design, while the processing was being carried out in a particular block, the others had no use whatsoever. It was only after the entire instruction had been processed, that the next instruction was fetched and executed. As a result, the minimum possible clock cycle time was very high. Besides this, if the instructions get more complex, the design might take multiple clock cycles due to the architecture design, which would further decrease the efficiency. To improve this issue, we introduce the concept of pipelining. This is based on the fact that the fetching of the next instruction does not require the complete processing of the previous one. As soon as an instruction is fetched and sent to the instruction decoding stage, we fetch the next one to start executing. In this process, each of the blocks within the architecture run parallel to each other performing different instructions on the same clock cycle. As a result, the clock cycle is significantly reduced.

Theoretical Analysis

The first step to making a pipelined processor is to divide the the entire execution of an instruction into different stages. In our case, we divide the process into five different stages, with one step per stage, as mentioned below:

1) IF: Instruction fetch from memory
2) ID: Instruction decode and register read
3) EX: EXecute operation or calculate address
4) MEM: Access memory operand
5) WB: Write result back to register

The idea is that each of these stages should be occupied in computation simultaneously on different instructions, so as to reduce the clock cycle time, to that of the worst-case time delay of a particular stage instead of the entire process. The speedup due to pipelining is given by the following formula:

$$T_{pipelined} = \frac{T_{sequential}}{N}$$

where

- $T_{pipelined}$ = Time between instructions in a pipeline implementation
- $T_{sequential}$ = Time between instructions in a non-pipeline implementation
- N = Number of stages

Note: This speed-up (as in the formula) is achieved only when the stages are balanced, i.e. all have the same time delay. The speed-up obtained will be lesser than the one given by the above formula for unbalanced stages.

Critical Analysis

Pipelining significantly speeds up the processor. However, there are many issues that we need to deal with while implementing this design. Resolving with all kinds of hazards (unlike structural, data and control can be resolved and are of interest to us) is of utmost importance in this process. The possible issues and the process of their resolution have been discussed below:

Data Hazards

A data hazard arises when an instruction depends on the completion of a data access by a previous instruction. For example, if the result of an instruction is being stored in a register or memory, and is being accessed in the next instruction. To resolve this issue, we add *pipeline registers* after each stage of the processor whose purpose is to maintain records (bookkeeping of a sort) in the circuit. Our aim is to store the results from various blocks along with the control signal at every stage on the registers. Besides this, we also save the write register rd on each of the successive registers to avoid write-back at some other register address. The use of storing data on registers is that we can directly access the values stored on the register, rather than waiting for it to be written in the memory or in some register. For example, if some computed data is needed in the next instruction, we can directly access this value, as present in the EX/MEM Register, rather than place a stall to access from the memory/register. In such a case, we need to choose from the two values (one present within the current clock cycle and one being taken from the register). We do this by checking for three things. First, we ensure that we actually have a write back causing a hazard. Then we check whether register rd is x0. In this case, we need not write back. Finally, we need to check if the address at rs1 (or rs2) within the ID/EX register is the same as that at rd within the EX/MEM register or the MEM/WB register. In these cases we need to use the updated values instead of the ones decoded by the new instruction.

We might also encounter **double data hazard**, when the value at some address is being used in successive instructions and being updated in each of

them. In this case, we need to use the most recent value stored on the pipeline register. We have the following conditions to ensure choosing data from from the available options:

- The MEM/WB stage has register write enabled (MEM/WB.RegWrite is true).
- The destination register in the MEM/WB stage is not register x0.
- Either there is no write-back happening in the EX/MEM stage to the same register, or the register being written in EX/MEM is different from the source register Rs1 in the ID/EX stage.
- The destination register in the MEM/WB stage matches the source register Rs1 in the ID/EX stage (MEM/WB.RegisterRd = ID/EX.RegisterRs1).

The above computations take place in the forwarding unit of the processor.

Another data hazard occurs in case of **load, followed by use** of the register. In this case, we have no option but to stall the pipeline for a clock cycle. Since by the time we need the write back, we have just calculated the address to access the data. Placing a stall allows to access the data from the MEM/WB register.

Control Hazards

A control hazard occurs when the processor encounters a branch instruction, like beq (Branch if Equal), and cannot determine the next instruction to fetch until the branch condition is evaluated. This uncertainty can lead to wasted cycles or incorrect execution of the instruction. The problem is that while the branch decision is made in the EX stage, the next instruction is already being fetched during IF stage (speculative execution). If the branch is taken, the fetched instruction is wrong, causing a control hazard.

Branch prediction helps mitigate control hazards by guessing the outcome of a branch instruction (such as BEQ) before evaluating the condition. The processor predicts branch not taken and continues fetching the next sequential instruction. In the event that the prediction was incorrect, we place a stall on the pipeline and restart the current instruction. We do this by sending ”zero” for each of the control signals ahead. As a result, nothing is computed on the current clock cycle. This check, along with that for load- use hazard takes place in the hazard detection unit.

Dealing with Double Data Hazards in Verilog

The code has been extensively tested on various test cases, one of which is given below (Hazards in the code are explained later in the report):

```
##TESTCASE 1
//memory has values 3,2,5 on
   successive locations
ld  x4  0(x0)  : 0
ld  x5  8(x0)  : 4
ld  x6  16(x0) : 8
beq x4  x5  label1[branch not taken]
add x5  x4  x5  : 16
beq x5  x6  label2
add x0  x0  x0  : 24
Sub x0  x0  x0  : 28
//two more random instructions which
    do not execute due to branching
Sub x7  x6  x4  : 40
AND x8  x7  x4  : 44
OR  x9  x4  x0  : 48
SD  x9  24(x0) : 52
```

Cycle-by-Cycle Operation Explanation

Cycle 1

- **IF:** Fetch the first instruction `ld x4, 0(x0)`.
- **ID, EX, MEM, WB:** Idle.

Cycle 2

- **IF:** Fetch the second instruction `ld x5, 8(x0)`.
- **ID:** Decode the first instruction; read register x0 and compute the effective address.
- **EX, MEM, WB:** Not active yet.

Cycle 3

- **IF:** Fetch the third instruction `ld x6, 16(x0)`.
- **ID:** Decode the second instruction; read x0.
- **EX:** First instruction computes its effective address $(0 + x0)$.
- **MEM, WB:** Not active yet.

Cycle 4

- **IF:** Fetch the fourth instruction `beq x4, x5, label1`.
- **ID:** Decode the third instruction; read x0.
- **EX:** Second instruction computes its effective address $(8 + x0)$.

- **MEM:** First instruction accesses memory using the computed address.
- **WB:** Not yet active.

Cycle 5

- **IF:** Fetch the fifth instruction `add x5, x4, x5`.
- **ID:** Decode the fourth instruction (branch); read x4 and x5.
- **EX:** Third instruction computes its effective address $(16 + x0)$.
- **MEM:** Second instruction accesses memory.
- **WB:** First instruction writes its loaded value into x4.

Cycle 6

- **IF:** Fetch the sixth instruction `beq x5, x6, label2`.
- **ID:** Decode the fifth instruction; read x4 and x5.
- **EX:** Fourth instruction (branch) evaluates the condition using x4 and x5.
- **MEM:** Third instruction accesses memory.
- **WB:** Second instruction writes its loaded value into x5. **The first branch is not taken, so the sequential flow continues.**

Cycle 7

- **IF:** Fetch the seventh instruction `add x0, x0, x0`.
- **ID:** Decode the sixth instruction (branch); read x5 and x6.
- **EX:** Fifth instruction executes the addition, using forwarded values if needed.
- **MEM:** Fourth branch instruction completes its branch resolution.
- **WB:** Third instruction writes its value into x6. **(Note: I7 is fetched on the assumption of sequential flow.)**

Cycle 8

- **IF:** Fetch the eighth instruction `sub x0, x0, x0`.
- **ID:** Decode the seventh instruction.
- **EX:** Sixth instruction (branch) evaluates its condition with updated values of x5 and x6. **Since the condition is true, the branch is taken.**
- **MEM:** Fifth instruction completes its arithmetic operation.
- **WB:** Fourth branch completes write-back (if applicable).
- At this point, the processor recognizes that I6's branch is taken. The instructions I7 and I8 that were fetched speculatively are now invalid and will be flushed

Cycle 9

- **IF:** Fetch the ninth instruction `Sub x7, x6, x4`.
- **ID:** Flushed instructions discarded.
- **EX, MEM, WB:** The pipeline continues processing with the new instruction stream starting at label2.

Cycle 10

- **IF:** Fetch the tenth instruction `AND x8, x7, x4`.
- **ID:** Decode the ninth instruction; prepare the subtraction operation.

Cycle 11

- **IF:** Fetch the eleventh instruction `OR x9, x4, x0`.
- **ID:** Decode the tenth instruction (AND); read x7 and x4.
- **EX:** Ninth instruction (subtraction for x7) executes.

Cycle 12

- **IF:** Fetch the twelfth instruction `SD x9, 24(x0)`.
- **ID:** Decode the eleventh instruction (OR); read x4 and x0.
- **EX:** Tenth instruction (AND) executes.
- **MEM:** Ninth instruction (subtraction) writes its result into x7.

Cycle 13

- **IF:** No new instruction is fetched as the pipeline begins to drain.
- **ID:** Decode the twelfth instruction (store); read x9.
- **EX:** Twelfth instruction computes its effective address, ie, 24 + x0.
- **MEM:** Prior arithmetic operations complete.
- **WB:** Earlier instructions finish write-back.

Cycle 14

- **IF/ID:** No new instructions.
- **EX:** The store instruction remains in EX if not already complete.
- **MEM:** The store writes the value from x9 to memory at address `24(x0)`.

- **WB:** Final write-back occurs for previous instructions.

Cycle 15

- **WB:** The store instruction completes its final stage, draining the pipeline.

Summary This timeline demonstrates that at any given clock cycle, each pipeline block works on a different instruction concurrently. Data dependencies are handled by forwarding data from later stages (MEM, WB) when needed, and control hazards are resolved in the EX stage when evaluating branch conditions. As seen, we *predict* branch not being taken in the first beq statement and the prediction is correct. As a result, stall is not needed. In the second case however, we flush the preloaded instructions since the prediction is incorrect this time. The new instructions are then executed as before.

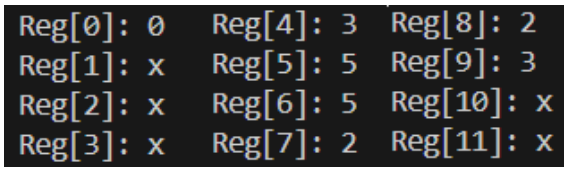

Figure 3. Final Register Values

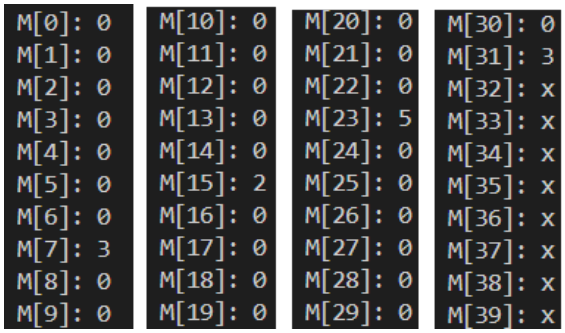

Figure 4. Final Memory Values

Results

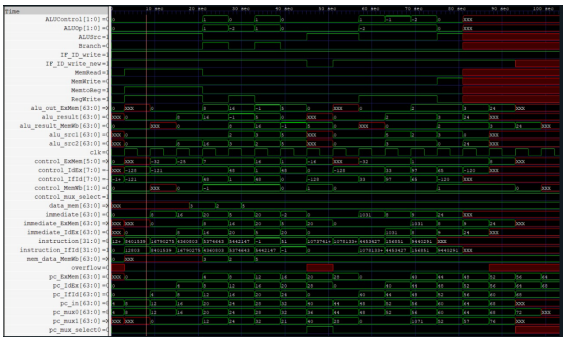

Figure 5. Simulation Result

Dealing with Double Data Hazards in Verilog

```
##TESTCASE 2
ld  x4  0(x0)
ld  x5  8(x0)
add x5  x4  x5
Sub x6  x5  x4
add x7  x5  x0
add x8  x7  x7
add x0  x4  x5
add x9  x8  x0
```

Cycle-by-Cycle Operation Explanation Below is a timeline that shows how the instructions flow through the five stages.

Cycle 1

- **IF:** Fetch `ld x4, 0(x0)`.
- **ID, EX, MEM, WB:** Idle.

Cycle 2

- **IF:** Fetch `ld x5, 8(x0)`.
- **ID:** Decode `ld x4, 0(x0)`; read register x0.
- **EX, MEM, WB:** Idle.

Cycle 3

- **IF:** Fetch `add x5, x4, x5`.
- **ID:** Decode `ld x5, 8(x0)`; read x0 for effective address computation.
- **EX:** `ld x4, 0(x0)` computes its effective address.
- **MEM, WB:** Idle.

Cycle 4

- **IF:** Fetch `Sub x5, x5, x4`.

- **ID:** Decode `add x5, x4, x5`; read registers x4 and x5.
- **EX:** `ld x5, 8(x0)` computes its effective address.
- **MEM:** `ld x4, 0(x0)` accesses memory.
- **WB:** Idle.

Cycle 5

- **IF:** Fetch `add x7, x5, x0`.
- **ID:** Decode `Sub x5, x5, x4`; read x5 and x4.
- **EX:** `add x5, x4, x5` performs the addition (producing a new value for x5).
- **MEM:** `ld x5, 8(x0)` accesses memory.
- **WB:** `ld x4, 0(x0)` writes its loaded value into x4.

Cycle 6

- **IF:** Fetch `add x8, x7, x7`.
- **ID:** Decode `add x7, x5, x0`; read registers x5 and x0.
- **EX:** `Sub x5, x5, x4` executes subtraction.
- **MEM:** `add x5, x4, x5`'s result is passed on (and potentially forwarded).
- **WB:** `ld x5, 8(x0)` writes its loaded value into x5.

Cycle 7

- **IF:** Fetch `add x0, x4, x5`.
- **ID:** Decode `add x8, x7, x7`; read register x7.
- **EX:** `add x7, x5, x0` executes addition.
- **MEM:** `Sub x5, x5, x4` proceeds to MEM.
- **WB:** `add x5, x4, x5` writes its result into x5 (if not already forwarded).

Cycle 8

- **IF:** Fetch `add x9, x8, x0`.
- **ID:** Decode `add x0, x4, x5`; read registers x4 and x5.
- **EX:** `add x8, x7, x7` executes its addition.
- **MEM:** `add x7, x5, x0` moves to MEM.
- **WB:** `Sub x5, x5, x4` writes its result into x5.

Cycle 9

- **IF:** No new instruction (pipeline draining).
- **ID:** Decode `add x9, x8, x0`; read registers x8 and x0.
- **EX:** `add x0, x4, x5` executes its addition.
- **MEM:** `add x8, x7, x7` proceeds to MEM.
- **WB:** `add x7, x5, x0` writes its result into x7.

Cycle 10

- **IF:** Pipeline continues to drain.
- **EX:** `add x9, x8, x0` executes.
- **MEM:** `add x0, x4, x5` proceeds to MEM.
- **WB:** `add x8, x7, x7` writes its result into x8.

Cycle 11

- **MEM:** `add x9, x8, x0` proceeds to MEM.
- **WB:** `add x0, x4, x5` writes its result (if applicable).

Cycle 12

- **WB:** `add x9, x8, x0` writes its result into x9.

Double Data Hazard and Its Resolution Consider the following subsequence:

```
add x5, x4, x5
Sub x5, x5, x4
add x7, x5, x0
```

Hazard Description

- **Data Dependency:** The instruction `add x5, x4, x5` updates register `x5`. Both `Sub x5, x5, x4` and `add x7, x5, x0` require the updated value of `x5` as an operand.
- **Double Data Hazard:** This is termed a "double" data hazard because two distinct instructions depend on the result produced by the same operation.

Resolution Technique Forwarding (Bypassing): In the implemented processor, the result computed by `add x5, x4, x5` in the EX stage is forwarded directly to the EX stages of the dependent instructions (`Sub x5, x5, x4` and `add x7, x5, x0`). This avoids waiting for the WB stage and eliminates the need for a stall. Also, we use the value of x5 updated in `Sub x5, x5, x4` in the following instruction instead of the former value to avoid the hazard.

Pipeline Impact: With effective forwarding, both dependent instructions receive the updated value of `x5`, and the pipeline can proceed without errors.

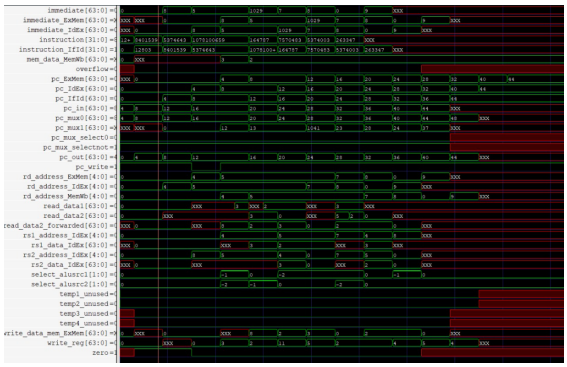

Figure 6. Simulation Result

Reg[0]: 0	Reg[7]: 2
Reg[1]: x	Reg[8]: 4
Reg[2]: x	Reg[9]: 4
Reg[3]: x	Reg[10]: x
Reg[4]: 3	Reg[11]: x
Reg[5]: 2	Reg[12]: x
Reg[6]: x	Reg[13]: x

Figure 7. Final Register Values

Result Plots

Dealing with Load Use Data Hazards in Verilog

```
##TESTCASE 3
ld  x4  0(x0)
ld  x5  8(x0)
add x6  x5  x4
sd x6 32(x0)
```

Cycle-by-Cycle Operation Explanation Below is a timeline that shows how the instructions flow through the five stages.

Cycle 1

- **IF:** Fetch `ld x4, 0(x0)`.
- **ID, EX, MEM, WB:** No active instructions yet.

Cycle 2

- **IF:** Fetch `ld x5, 8(x0)`.
- **ID:** Decode `ld x4, 0(x0)` and read `x0`.
- **EX, MEM, WB:** Remain idle for this instruction.

Cycle 3

- **IF:** Fetch `add x6, x5, x4`.
- **ID:** Decode `ld x5, 8(x0)` and read `x0`.
- **EX:** `ld x4, 0(x0)` computes its effective address.
- **MEM, WB:** Awaited for earlier instructions.

Cycle 4

- **IF:** Fetch `sd x6, 32(x0)`.
- **ID:** Decode `add x6, x5, x4` and attempt to read `x5` and `x4`.
- **EX:** `ld x5, 8(x0)` calculates its effective address.
- **MEM:** `ld x4, 0(x0)` accesses memory to load the data.
- **WB:** Not active yet.

Cycle 5

- **Stall Inserted: Because the `add` instruction depends on the results of the loads, the processor inserts a stall (a bubble) to delay its progress.** The `add` instruction remains in the ID stage for an extra cycle.
- **EX:** `ld x5, 8(x0)` moves to its MEM stage (or completes its address calculation and begins memory access).
- **MEM:** `ld x4, 0(x0)` moves toward WB, finishing its memory access.
- **WB:** `ld x4, 0(x0)` writes its loaded data into `x4`.

Cycle 6

- **IF:** `sd x6, 32(x0)` continues its flow (now in the ID stage in the next cycle).
- **ID:** The stalled `add x6, x5, x4` is still waiting; now, the loaded values are either in the MEM stage or being written back.
- **EX:** With the stall resolved, the `add` instruction moves to the EX stage and performs its addition. Forwarding paths deliver the freshly loaded values from `x4` (from WB of `ld x4`) and `x5` (from MEM or WB of `ld x5`) as needed.
- **MEM:** `ld x5, 8(x0)` proceeds to its WB stage soon.
- **WB:** `ld x5, 8(x0)` writes its value into `x5`.

Cycle 7

- **IF/ID:** `sd x6, 32(x0)` continues through the pipeline.
- **EX:** `add x6, x5, x4` proceeds to the MEM stage (its arithmetic result was computed in EX).
- **WB:** The result of `add x6, x5, x4` is written back into `x6`.

Cycle 8

- **ID:** `sd x6, 32(x0)` decodes and reads `x6` (now containing the correct result from the add).
- **EX:** `sd x6, 32(x0)` calculates the effective address (`x0 + 32`).
- **MEM:** `sd x6, 32(x0)` writes the value from `x6` to memory.
- **WB:** Although stores do not update registers, this stage completes the store operation.

Explanation of the Load–Use Hazard

- The `add x6, x5, x4` instruction depends on the data loaded by the previous two `ld` instructions.
- In a pipelined processor, the loaded data is not available immediately in the cycle following the load (it becomes available only after the MEM or WB stage).
- **To resolve this hazard, the pipeline detects that the add operands are not yet ready and inserts a stall.** This extra cycle delays the `add` until the loaded values are available—either forwarded from the MEM/WB stages or after the load instructions have completed.
- This stall (or bubble) temporarily holds the dependent instruction in the ID stage, ensuring that when it enters EX, it operates on valid, up-to-date data. Without this stall, the `add` could execute with incorrect (or old) values, leading to erroneous results.

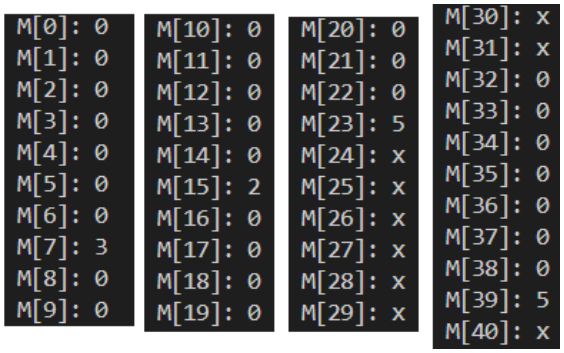

Figure 10. Final Memory Values

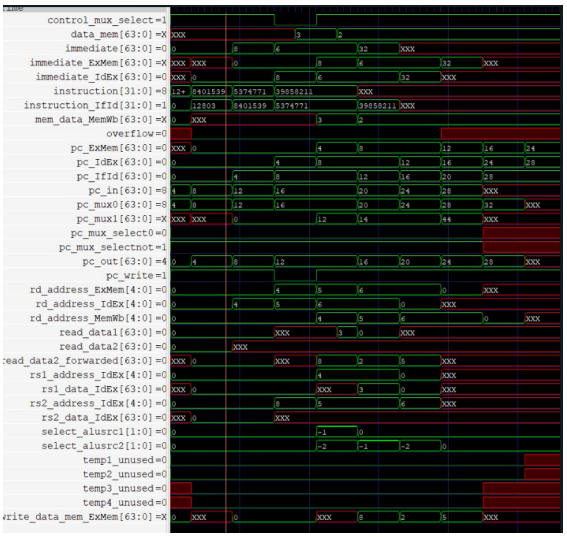

Figure 8. Simulation Result

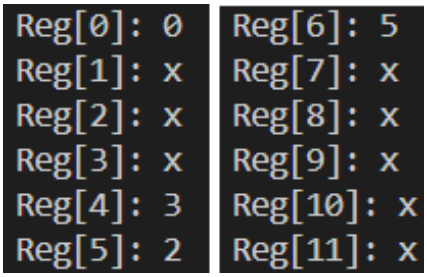

Figure 9. Final Register Values

Result Plots

Individual Contributions

Sequential Design

- Pa.Kiruba: Immediate generator and Instruction Fetch Block
- Krrish Goenka: Data & Instruction Memory and Register File
- Sajiv Singh: Control Unit

Pipeline Design

- Pa.Kiruba: EX and MEM Blocks
- Krrish Goenka: IF and ID Blocks
- Sajiv Singh: WB Block, Forwarding Unit and Hazard Detection Unit